

Recall Project

Presentation sections

Problem & Motivation

What problem are you solving?

Finding and sending specific media in a chat is unnecessarily hard

- The built-in chat media picker is just a chronological camera roll with no search
 - If the file is more than a few weeks old, you are scrolling blind with $O(n)$ time complexity linear search
- Memes and downloaded media have no metadata to anchor them. no date context, no location, no filename
- Google Photos has semantic search but breaks down on anything more complex than a single entity query
- And using Google Photos still forces a context switch and adds many steps: leave the chat, open a separate app, search there, share back, find the conversation again

Who experiences this problem?

- Anyone who regularly shares media in chat and has a library that has grown over multiple years
- People who save memes, reaction images, and downloaded media with no consistent naming or organization

Why does it matter in real life?

- Media sharing is one of the most frequent actions in mobile messaging
- When retrieval takes too long, users either give up or send something less relevant
- The problem is not occasional. Anyone with a large library runs into this regularly.

Target Users & Context

Who are your target users?

People who frequently share media (photos, videos, memes) esp. in chat applications. More specifically:

- Users with large, accumulated media libraries (photos from multiple years, hundreds of downloaded memes, saved videos)
- Users who remember what they are looking for conceptually but not precisely. They have a vague memory, not an exact filename or date
- Mobile-first users who do most of their sharing inside messaging apps like iMessage, KakaoTalk, Instagram DMs, or WhatsApp

In what situation/context does the problem occur ?

- Always mid-conversation, under mild time pressure
- The user is actively chatting and wants to send something specific to fit the current moment
- The retrieval need is reactive: something in the conversation triggers the memory of a file they want to send
- The longer retrieval takes, the less relevant the file becomes to the conversation

Persona or short scenario regarding the target user

- User A is recapping a trip with a friend over chat. Remembers a video of themselves swimming at a waterfall from two years ago and wants to send it. Opens the media picker, sees only recent photos, starts scrolling, gives up. The conversation moves on before they find it
- User B is mid-conversation and wants to send a specific meme they downloaded four months ago. Remembers it was something like "truth nuke" but cannot recall the exact name or the character in it. The media picker has no search. They scroll, find nothing, and settle for sending a different meme instead

Proposed Idea / Solution Concept

What is your system/app/product and what does it do?

- Recall is an AI media retrieval assistant that lives inside the keyboard row of any chat app, opening as a bottom sheet without leaving the conversation
- Users type a natural language description of what they remember and Recall surfaces matching photos, videos, and memes from their library
- Users select files directly from the results and send them back to the chat in one continuous flow, never switching apps

What makes it different or better than existing solutions?

- The chat media picker has no search at all. Recall replaces blind scrolling with natural language querying
- Google Photos handles single-word searches but fails on multi-token semantic queries like "waterfall video swimming"
- Google Photos requires leaving the conversation entirely. Recall lives in the keyboard so the user never leaves the chat
- When the user's own memory of the query is wrong, Recall offers "Did you mean..." reformulations rather than leaving them at a dead end

2-4 main features of your system & why did you design it this way?

User Testing Plan

Just a rough testing plan is enough. At minimum, you should cover:

Who will you test your system with? (target users)

What tasks will users perform? (no need to be too detailed at this stage)

How will you measure success? (e.g., usability, satisfaction)

Project Timeline (Weekly Progress Plan)

Figma Prototype

Flow 1: photo retrieval in chat.

Scenario: The user wants to find travel photos and videos from about two years ago to share in a conversation.

1. User opens Recall
 - a. User is in a chat app. They tap the Recall icon in the keyboard row. Recall opens as a bottom sheet. with a search bar auto-focused and the keyboard already up. Below the search bar, the user sees two elements:
 - i. A history button (clock icon) to the left of the search bar, tapping which reveals a dropdown list of recent searches (e.g., "beach sunset", "mom birthday", "dog video"). These are tappable to immediately re-run that query.
 - ii. A row of recent media thumbnails below, showing recently accessed or shared files from previous Recall sessions.
2. User types "waterfall"
 - a. As the user types, the search bar shows autofill suggestions in a dropdown directly beneath it, functioning similarly to Google Search autocomplete. Suggestions are ranked by a combination of:
 - i. The user's own past searches (pulled from history, shown with a clock icon)
 - ii. Semantically related completions based on what's in their library (e.g., "waterfall video", "waterfall hiking", "waterfall swimming")
 - iii. General popular completions for that prefix (shown with a search icon, no clock)
3. User hesitates mid-type
 - a. After typing "waterfall", the user pauses for roughly a half second without selecting an autofill suggestion. Recall interprets this as intent to search and begins pre-fetching results in the background. The autofill dropdown remains visible but results start loading beneath it. If the user resumes typing, results update. If they stop fully, the dropdown closes and results surface.
4. Results appear for "waterfall"
 - a. Autofill dropdown dismisses. Results grid appears: a mix of photos and videos from the user's library that match "waterfall" semantically. Videos are indicated by a play icon overlay and a duration badge in the bottom-right corner of the thumbnail, same as iOS Photos.
5. User continues typing: "waterfall video swimming"
 - a. The autofill dropdown reappears immediately on keystroke. As the query grows, suggestions narrow. Library-derived completions now include things like "waterfall swimming video 2023." Results grid updates in real time as

tokens are added. Thumbnails that no longer match fade and collapse, new matches appear.

6. User finds the correct video and taps on it to select it.
 - a. A checkmark appears in the top-right corner of the video card. A selection tray appears at the bottom of the screen showing the selected item's thumbnail, a count badge ("1 selected"), and a Send button. The user can tap additional thumbnails to add them to the selection.
7. User long-presses the video card to open fullscreen
 - a. Long press opens the fullscreen video viewer. The video autoplays muted. Scrubber at the bottom. Back arrow top-left. The checkmark state is preserved.
8. User taps the options icon in the top-right of the fullscreen view
 - a. Same three-dot icon placement as Google Photos' fullscreen viewer
 - b. brings out action sheet with these options. (subject to change)
 - i. Find similar
 - ii. Find from date
 - iii. Copy
 - iv. Save to camera roll
 - v. Share via...
 - vi. Add to album
9. User taps "Find from the same date"
 - a. Recall runs a new query scoped to the date metadata of the selected video. The fullscreen viewer closes and the results grid reappears
 - b. The grid reorganizes with a date header (e.g., "August 14, 2023") grouping everything taken that day. Same grouped-by-date pattern as iOS Photos and Google Photos. The original video remains checked in the selection tray at the bottom.
10. User taps another photo from the trip
 - a. Single tap adds it to the selection. Checkmark appears on that thumbnail. Selection tray updates to "2 selected" and shows both thumbnails.
11. user sends message with both the video and the photo
 - a. User taps Send in the selection tray. Both files are passed back to the chat. Recall closes.



Flow 2: Query Refinement

Scenario: The user vaguely remembers downloading a meme similar to "truth nuke" about four months ago but doesn't remember the exact name or subject character.

1. User opens Recall
 - a. Same entry point as Flow 1. Search bar auto-focused. History dropdown available via the clock icon.

2. User types "truth nuke gif"
 - a. Autofill suggestions appear. Library-derived suggestions include things like "truth nuke wojak", "truth nuke meme". User ignores them and submits the query as-is (taps Search or hits return).
3. Results appear: only wojak variants
 - a. The results grid shows about 20–30 Wojak-related truth nuke images. The user scans them. The one they're thinking of isn't there. They scroll down.
4. User reaches the bottom of the capped results list
 - a. Results are capped at 50 by default. At the bottom of the list, two persistent options appear as a footer card:
 - i. "Show more results" a text button that loads the next 20
 - ii. "Did you mean..." a suggestion chip that indicates Recall has alternative query interpretations available
5. User taps "Show more results"
 - a. 20 additional results load and append to the grid. The footer card reappears at the new bottom. User scrolls through the new results. Still not there.
6. User reaches bottom again, same two options
 - a. User decides the result doesn't exist under this query formulation. They tap "Did you mean..."
7. "Did you mean..." panel opens
 - a. A bottom sheet slides up showing a list of alternate query suggestions. Each suggestion is a reformulation of the original query based on:
 - i. Known meme name aliases
 - ii. Combinations with entities Recall detected in the user's library
 - b. The list looks something like:
 - i. trvth nuke gif
 - ii. trvth nuke sayaka
 - iii. nuke reaction image
8. User spots "trvth nuke sayaka"
 - a. Seeing "sayaka" triggers the memory. The meme had that character. The user taps that row.
9. Query updates, correct result found
 - a. The search bar updates to "trvth nuke sayaka" and results reload. The target meme appears in the first few results. User taps it, it opens fullscreen. They confirm it's the right one, then send it.

UX specs

UI specs

- gotta decide color palette?
- fonts?
-